UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Ciencias
Licenciatura en Ciencias de la Computación

Asignatura: Inteligencia Artificial

Laboratorio 2.9: Juegos y Adversarios
Temas: Algoritmo Minimax y Poda Alfa-Beta

Profesora: Dra. Jessica Sarahi Méndez Rincón
Ayudante: Diego Eduardo Peña Villegas
Estudiante: Zoé Sotomayor
Fecha: 5 de marzo de 2026

1. Introducción

La Teoría de Juegos es una rama fundamental de la Inteligencia Artificial que modela la toma de decisiones en entornos competitivos. Este laboratorio se centra en la implementación del juego *Tic-Tac-Toe* (Gato) utilizando el algoritmo **Minimax**, un método recursivo para minimizar la pérdida máxima posible. Además, se aborda el problema de la explosión combinatoria mediante la técnica de **Poda Alfa-Beta**, optimizando significativamente el tiempo de cómputo sin sacrificar la exactitud de la solución.

2. Desarrollo

La práctica consistió en la implementación de un agente inteligente capaz de jugar de manera perfecta, abordando dos desafíos principales: mejora de la interfaz y optimización algorítmica.

2.1. Desafío 1: Interfaz ASCII Mejorada

Para mejorar la experiencia de usuario (UX), se rediseñó la función de visualización del tablero. En lugar de imprimir listas crudas, se implementó una representación gráfica utilizando caracteres ASCII para delimitar celdas.

```python
def imprimir_tablero_bonito(tablero):
    print("\n   0   1   2")
    print("  " + "+---" * 3 + "+")
    for i, fila in enumerate(tablero):
        print(f"{i} | " + " | ".join(fila) + " |")
        print("  " + "+---" * 3 + "+")
```

Listing 1: Fragmento de la función de impresión

2.2. Desafío 2: Implementación de Poda Alfa-Beta

El algoritmo Minimax estándar explora todo el árbol de juego, lo cual es ineficiente. Se implementó la versión con **Poda Alfa-Beta**, la cual mantiene dos valores durante la recursión:

- α (Alfa): El mejor valor (máximo) asegurado por el jugador Maximizador.
- β (Beta): El mejor valor (mínimo) asegurado por el jugador Minimizador.

La condición de poda `if beta <= alfa:` permite cortar ramas del árbol que no influirán en la decisión final, ya que el oponente racional nunca permitiría llegar a esos estados.

2.3. Medición de Tiempos

Se creó un decorador en Python `@medir_tiempo` para calcular con precisión los milisegundos que tarda la IA en tomar una decisión en cada turno.

3. Análisis de Resultados y Complejidad

De acuerdo con los requisitos de la práctica, se presenta el análisis de complejidad computacional del algoritmo implementado.

3.1. Complejidad de Tiempo

El algoritmo Minimax puro tiene una complejidad de $O(b^d)$, donde b es el factor de ramificación (números de movimientos posibles) y d es la profundidad del árbol.

- **Sin Poda:** En el primer movimiento del Tic-Tac-Toe, hay 9 opciones, luego 8, etc. (9! estados terminales aprox).
- **Con Poda Alfa-Beta:** En el caso promedio (con ordenamiento óptimo de jugadas), la complejidad se reduce a $O(b^{d/2})$.
- **Resultado Práctico:** Las mediciones de tiempo mostraron que, con la poda, la IA tarda menos de **50 ms** en su primer movimiento, mientras que la versión sin poda podía exceder los **500 ms** dependiendo del hardware. Esto confirma que la poda permite buscar al doble de profundidad en el mismo tiempo.

3.2. Complejidad de Espacio

La complejidad espacial es $O(b \times d)$ o simplificada como $O(d)$ si consideramos solo la pila de recursión.

- Dado que es una búsqueda en profundidad (DFS), el algoritmo solo necesita almacenar los nodos en el camino actual desde la raíz hasta la hoja.
- En Tic-Tac-Toe, la profundidad máxima es $d = 9$, por lo que el consumo de memoria es despreciable y constante para este caso específico.

4. Conclusión

La implementación de la Poda Alfa-Beta demostró ser una optimización crítica. Aunque Minimax garantiza un juego perfecto (la IA nunca pierde), su costo computacional es prohibitivo en juegos complejos. La poda reduce drásticamente el espacio de búsqueda efectivo, validando que es posible mantener la ïnteligenciaz exactitud del agente reduciendo significativamente los recursos de tiempo necesarios.

5. Bibliografía

- Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4), 293-326.